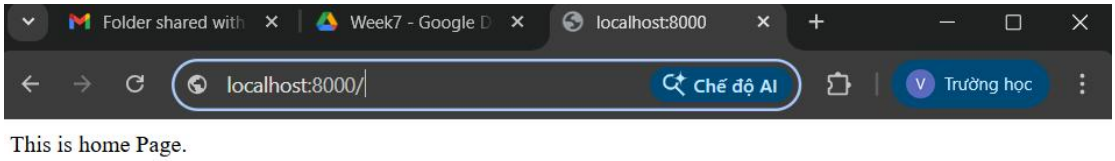
Name:Vương Bá Phục ID:GCS230123

ScreenShot 1:

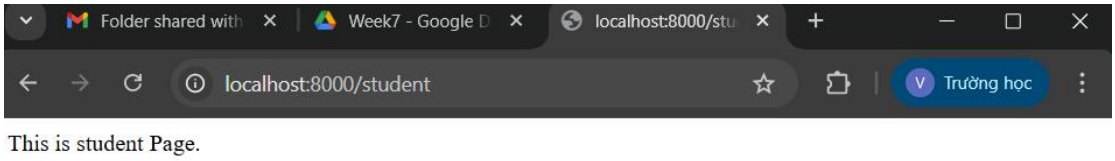
```
3
4
5

C:\COMP1842\npmtest>node server.js
C:\COMP1842\npmtest\server.js:2
cost server = http.createServer(function (req, res) {   //creating server
     ^^^^^^

SyntaxError: Unexpected identifier 'server'
    at wrapSafe (node:internal/modules/cjs/loader:1743:18)
    at Module._compile (node:internal/modules/cjs/loader:1786:20)
    at Object..js (node:internal/modules/cjs/loader:1943:10)
    at Module.load (node:internal/modules/cjs/loader:1533:32)
    at Module._load (node:internal/modules/cjs/loader:1335:12)
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.14.0

C:\COMP1842\npmtest>node server.js
Node.js web server at port 8000 is running..
^C
C:\COMP1842\npmtest>node server.js
Node.js web server at port 8000 is running..
^C
C:\COMP1842\npmtest>node server.js
Node.js web server at port 8000 is running..
```

ScreenShot 2:



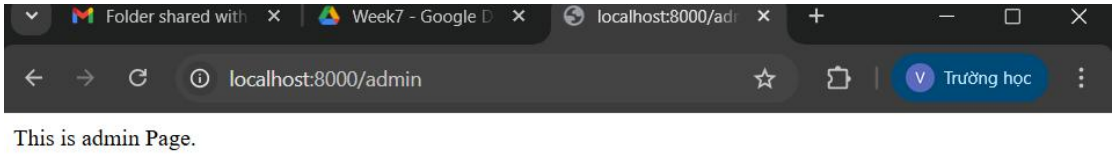
This is home Page.

ScreenShot 3:



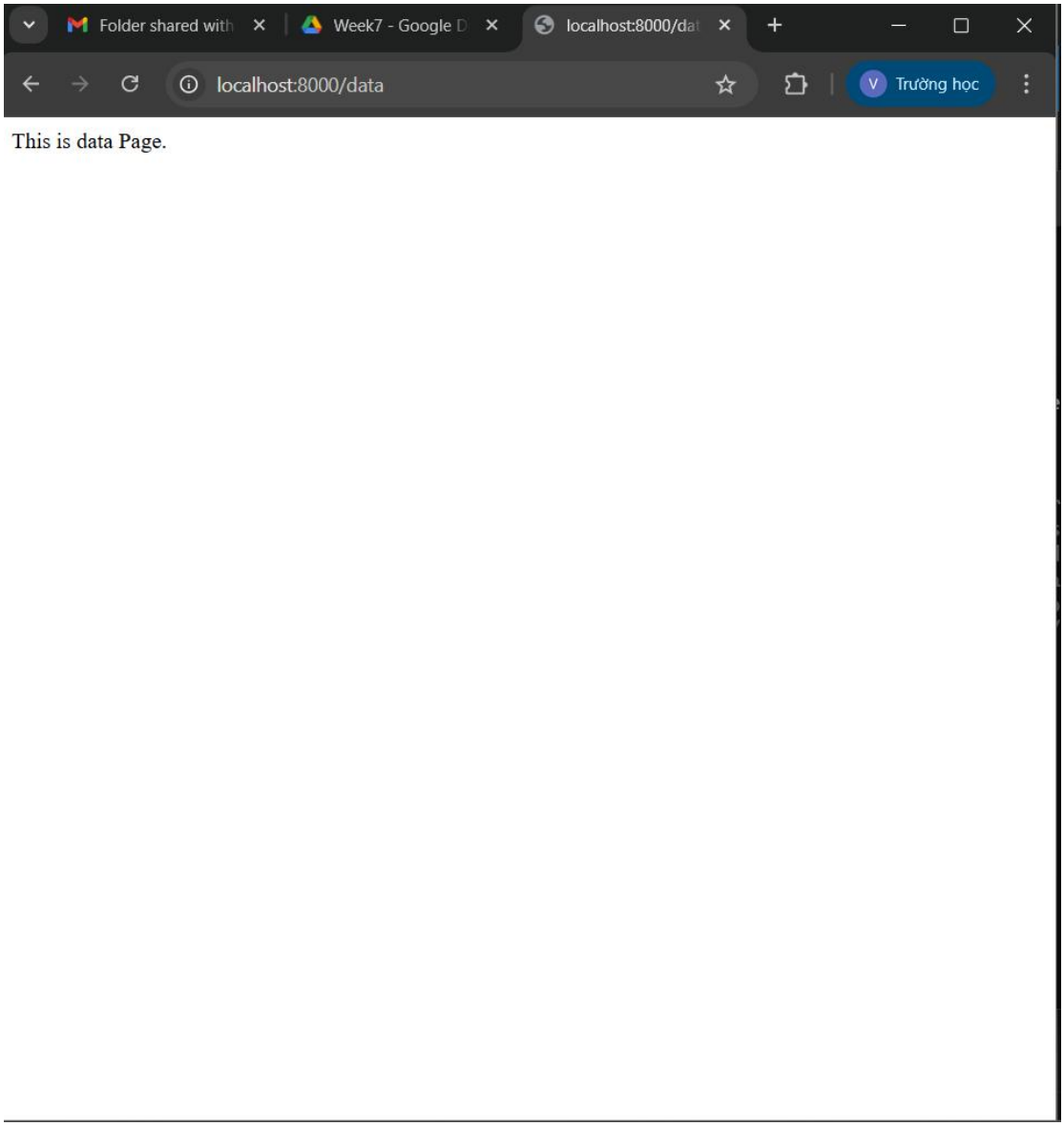
This is student Page.

ScreenShot 4:

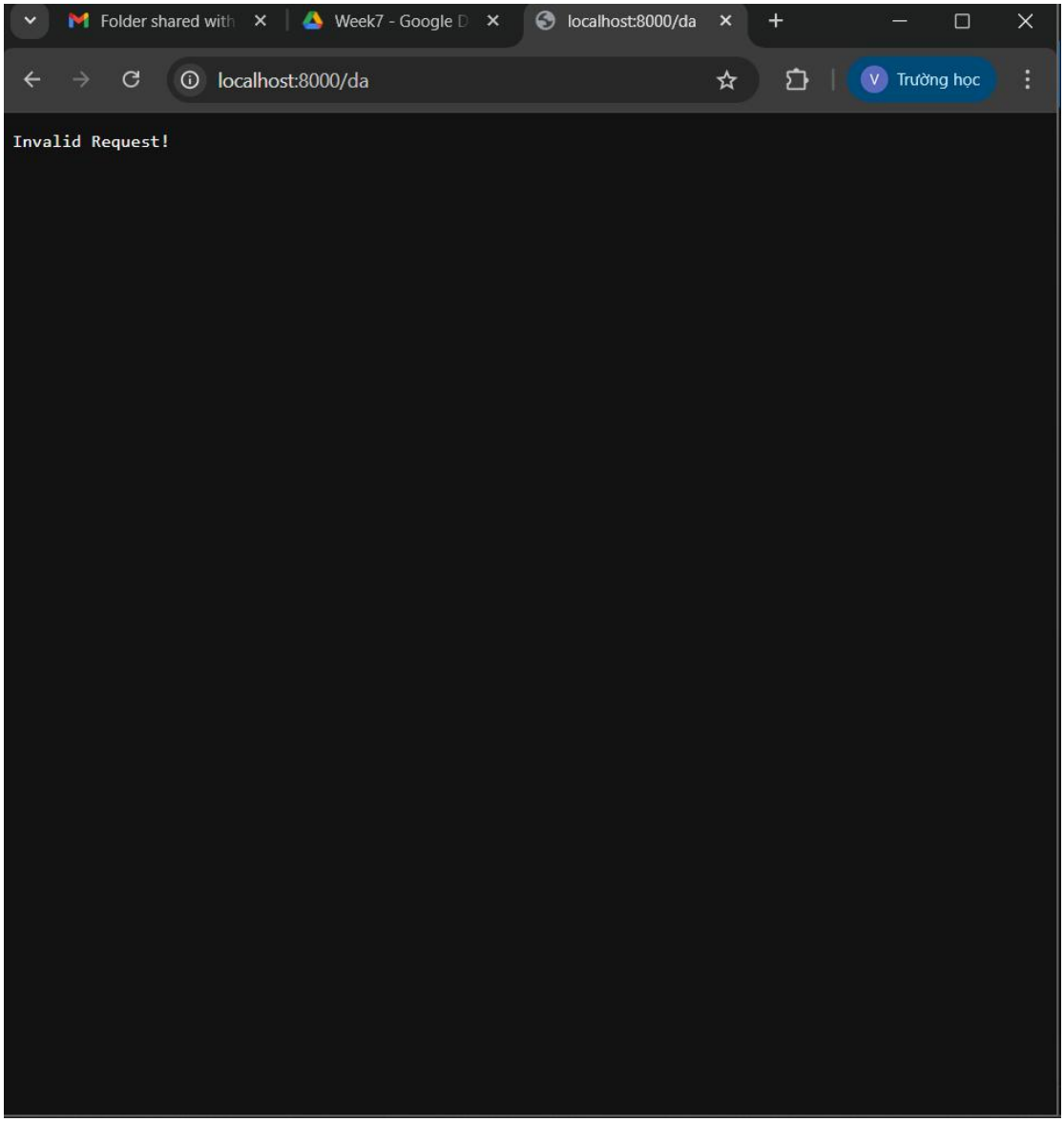


This is admin Page.

ScreenShot 5:



ScreenShot 6:



UNDERSTANDING:

The Node.js server example demonstrates how Node.js can be used to create a simple web server that responds to HTTP requests. Unlike traditional server technologies such as PHP or ASP.NET, Node.js works using a **single-threaded, non-blocking, asynchronous model**, which allows it to handle multiple requests efficiently without waiting for tasks such as file reading to complete before processing the next request.

The code begins by importing the built-in **HTTP module** using the `require()` function. This module provides the functionality required to create a web server and communicate over the HTTP protocol. Once the module is imported, the `http.createServer()` method is used to create the server. Inside this method, a callback function is defined which receives two parameters: `request` and `response`. These objects

represent the incoming client request and the server's response that will be sent back to the browser.

The `request` object contains useful information about the client's request, such as the requested URL, headers, and other metadata. In the example, the `req.url` property is used to check which page the user is trying to access. Conditional statements such as `if` and `else if` allow the server to return different responses depending on the URL path. For example, if the user visits `/home`, `/student`, or `/admin`, the server sends different HTML responses.

The `response` object is used to send data back to the client. The `writeHead()` method sets the HTTP header, including the status code and content type (for example `text/html` or `application/json`). After that, the `write()` method sends the content of the response, and the `end()` method finishes the response so the browser knows the data transfer is complete.

Finally, the server listens on **port 8000**, which allows users to access it through a browser by navigating to `localhost:8000`. This demonstrates how Node.js can be used to create a simple server capable of returning both HTML and JSON responses, forming the basic structure of a REST API or web application backend.